

siêu mọ cho lambda

MẸO HỌC FUNCTIONAL INTERFACE TRONG JAVA (BẢNG ĐẦY ĐỦ)

> **Cách dùng bảng này:** Mỗi interface đều có 1 mẹo nhỏ + ví dụ hàm lambda tương ứng.
> Học theo **màu sắc và quy tắc** sẽ nhớ lâu hơn học thuộc lòng.

1. NHÓM CORE (4 cái cha – nhớ là đủ)

Functional interface	Mẹo nhớ	Lambda ví dụ	Input → Output
Supplier<T>	Cung cấp – không vào, chỉ ra	() -> "Hello"	() → T
Consumer<T>	Tiêu thụ – vô mà không ra	s -> System.out.println(s)	T → void
Predicate<T>	Phán đoán – trả về đúng/sai	x -> x > 0	T → boolean
Function<T,R>	Biến đổi – vào kiểu này ra kiểu kia	s -> s.length()	T → R

2. NHÓM "BI" (2 tham số đầu vào)

Functional interface	Mẹo nhớ	Lambda ví dụ
BiConsumer<T,U>	2 tham số – không trả về	(a,b) -> System.out.println(a+b)
BiPredicate<T,U>	2 tham số – trả về boolean	(s,i) -> s.length() > i
BiFunction<T,U,R>	2 tham số – biến đổi thành R	(s,i) -> s.substring(0,i)
BinaryOperator<T>	2 tham số cùng kiểu – ra cùng kiểu	(a,b) -> a + b
UnaryOperator<T>	1 tham số – vào ra cùng kiểu	x -> x * x

 BinaryOperator = BiFunction<T,T,T>
 UnaryOperator = Function<T,T>

3. NHÓM NGUYÊN THỦY (Int, Long, Double) – KHÔNG DÙNG GENERIC

📌 Supplier (không vào, có ra)

Interface	Trả về	Lambda
<code>IntSupplier</code>	<code>int</code>	<code>() -> 100</code>
<code>LongSupplier</code>	<code>long</code>	<code>() -> 100L</code>
<code>DoubleSupplier</code>	<code>double</code>	<code>() -> 3.14</code>
<code>BooleanSupplier</code>	<code>boolean</code>	<code>() -> true</code>

📌 Consumer (chỉ tiêu thụ)

Interface	Nhận vào	Lambda
<code>IntConsumer</code>	<code>int</code>	<code>x -> System.out.println(x)</code>
<code>LongConsumer</code>	<code>long</code>	<code>x -> System.out.println(x)</code>
<code>DoubleConsumer</code>	<code>double</code>	<code>x -> System.out.println(x)</code>

📌 Predicate (trả về boolean)

Interface	Nhận vào	Lambda
<code>IntPredicate</code>	<code>int</code>	<code>x -> x > 0</code>
<code>LongPredicate</code>	<code>long</code>	<code>x -> x > 0L</code>
<code>DoublePredicate</code>	<code>double</code>	<code>x -> x > 0.0</code>

📌 Function (biến đổi)

Interface	Input	Output	Lambda
<code>IntFunction<R></code>	<code>int</code>	<code>R</code>	<code>i -> "number " + i</code>
<code>LongFunction<R></code>	<code>long</code>	<code>R</code>	<code>l -> Long.toString(l)</code>
<code>DoubleFunction<R></code>	<code>double</code>	<code>R</code>	<code>d -> d * 2</code>

UnaryOperator (vào = ra, cùng kiểu)

Interface	Input/Output	Lambda
IntUnaryOperator	<code>int → int</code>	<code>x -> x + 1</code>
LongUnaryOperator	<code>long → long</code>	<code>x -> x * 2L</code>
DoubleUnaryOperator	<code>double → double</code>	<code>x -> x / 2</code>

BinaryOperator (2 tham số, cùng kiểu)

Interface	Input/Output	Lambda
IntBinaryOperator	<code>(int,int) → int</code>	<code>(a,b) -> a + b</code>
LongBinaryOperator	<code>(long,long) → long</code>	<code>(a,b) -> a * b</code>
DoubleBinaryOperator	<code>(double,double) → double</code>	<code>(a,b) -> Math.max(a,b)</code>

4. NHÓM "TO..." (CHUYỂN ĐỔI SANG NGUYÊN THỦY)

Interface	Input	Output	Mẹo nhớ
<code>ToIntFunction<T></code>	<code>T</code>	<code>int</code>	Biến T thành int
<code>ToLongFunction<T></code>	<code>T</code>	<code>long</code>	Biến T thành long
<code>ToDoubleFunction<T></code>	<code>T</code>	<code>double</code>	Biến T thành double
<code>ToIntBiFunction<T,U></code>	<code>T,U</code>	<code>int</code>	2 tham số → int
<code>ToLongBiFunction<T,U></code>	<code>T,U</code>	<code>long</code>	2 tham số → long
<code>ToDoubleBiFunction<T,U></code>	<code>T,U</code>	<code>double</code>	2 tham số → double
<code>IntToLongFunction</code>	<code>int</code>	<code>long</code>	int → long
<code>IntToDoubleFunction</code>	<code>int</code>	<code>double</code>	int → double
<code>LongToIntFunction</code>	<code>long</code>	<code>int</code>	long → int
<code>LongToDoubleFunction</code>	<code>long</code>	<code>double</code>	long → double
<code>DoubleToIntFunction</code>	<code>double</code>	<code>int</code>	double → int
<code>DoubleToLongFunction</code>	<code>double</code>	<code>long</code>	double → long

5. NHÓM "OBJ" (OBJECT + NGUYÊN THỦY)

Interface	Nhận vào	Mẹo nhanh
<code>ObjIntConsumer<T></code>	<code>(T, int)</code>	Object + int → void
<code>ObjLongConsumer<T></code>	<code>(T, long)</code>	Object + long → void
<code>ObjDoubleConsumer<T></code>	<code>(T, double)</code>	Object + double → void

Ví dụ: `ObjIntConsumer<String> c = (s, i) -> System.out.println(s + i);`

BẢNG TỔNG KẾT MẸO NHỚ NHANH (NHÌN LÀ NHỚ)

Tên	Nhận vào	Trả ra	1 câu thần chú
Supplier	không	có	<i>Bán hàng – chỉ đưa ra</i>
Consumer	có	không	<i>Ăn hết – không thải ra</i>
Predicate	có	boolean	<i>Hỏi thật – đúng hay sai</i>
Function	có	có	<i>Pha chế – vào gì ra nấy</i>
Operator	1 hoặc 2	cùng kiểu	<i>Sao chép – vào ra một giống</i>
Bi...	2 tham số	(tùy)	<i>Song kiếm – hai món vào</i>
To...	(tùy)	int/long/double	<i>Chuyển tiền – sang nguyên thủy</i>
Obj...	Object + nguyên thủy	void	<i>Kết hợp – ông + số</i>
...Supplier	không	kiểu nguyên thủy	<i>ATM – chỉ nhà số cụ thể</i>

mẹo cho stream

1. Công thức thần thánh của Stream

STREAM = SOURCE → INTERMEDIATE → TERMINAL

Nguồn dữ liệu



Các bước xử lý



Kết quả cuối



Ví dụ:

`</> Java`



```
numbers.stream()
    .filter(n -> n > 5)
    .count();
```

Phần

Ý nghĩa

`numbers.stream()`

lấy stream

`filter(...)`

xử lý trung gian

`count()`

kết thúc + trả kết quả

2. Mẹo nhớ bản chất Stream

Stream KHÔNG lưu dữ liệu

Collection giống cái hộp chứa đồ.

Stream giống ống nước.

List ---> Stream ---> xử lý ---> kết quả



Stream chỉ cho data chảy qua.

Intermediate Operations

Methods involving intermediate operations:

- `filter()` returns a new stream containing elements that match a predicate;
- `limit()` returns a new stream containing the first `n` elements of the stream;
- `skip()` returns a new stream without the first `n` elements of the stream;
- `distinct()` returns a new stream containing only the unique elements resulting from `equals` ;
- `sorted()` returns a new stream containing elements sorted in the natural order or **comparator** passed to it;
- `peek()` returns the same stream of elements but allows you to observe the current elements of the stream for debugging;
- `map()` returns a new stream containing the elements obtained by applying a function (i.e., transforming each element).

Bảng tổng hợp				
Method	Ý nghĩa	Mental model	Input → Output	
filter	lọc	cái rây	Stream → Stream	
map	biến đổi	biến hình	Stream → Stream	
sorted	sắp xếp	xếp hàng	Stream → Stream	
distinct	unique	xóa clone	Stream → Stream	
limit	lấy N đầu	chặn đầu	Stream → Stream	
skip	bỏ N đầu	nhảy qua	Stream → Stream	
peek	nhìn lên	debug cam	Stream → Stream	

Terminal Operations

Terminal Operations:

- `count()` returns the number of elements in the stream as a `long` value;
- `max()/min()` returns the optional largest/smallest element of the stream according to the given comparator;
- `reduce()` combines values from a stream into a single value (a composite value);
- `findFirst()` / `findAny()` returns the first/any element of the stream as `Optional` ;
- `anyMatch()` returns `true` if at least one element matches a `Predicate` (see also: `allMatch` , `noneMatch`);
- `forEach` takes a **consumer** and applies it to each element of the stream (e.g., prints it);
- `collect()` returns a collection of values in the stream;
- `toArray()` returns an array of values in a stream.

Bảng tổng hợp

Method	Ý nghĩa	Mental model	Return
<code>count</code>	đếm	calculator	<code>long</code>
<code>collect</code>	gom	đóng hộp	<code>List/Set/...</code>
<code>reduce</code>	gộp	combine	<code>value</code>
<code>forEach</code>	duyệt	loop	<code>void</code>
<code>max/min</code>	cực trị	tìm boss	<code>Optional</code>
<code>findFirst</code>	lấy đầu	first item	<code>Optional</code>
<code>findAny</code>	lấy đại	random item	<code>Optional</code>
<code>anyMatch</code>	có ít nhất 1?	OR	<code>boolean</code>
<code>allMatch</code>	tất cả đúng?	AND	<code>boolean</code>
<code>noneMatch</code>	không ai đúng?	NOT	<code>boolean</code>
<code>toArray</code>	đổi array	export	<code>array</code>

Operations such as filter, map, reduce... are called higher-order functions because they accept other functions as arguments.

nghĩa là:

Các hàm như `filter`, `map`, `reduce`, `forEach`, `anyMatch` ... được gọi là **higher-order functions** vì chúng nhận một hàm khác làm tham số.

Ví dụ:

```
Java
list.stream()
    .filter(x -> x > 5)
```

Ở đây:

```
Java
x -> x > 5
```

là một function (lambda expression).

Và:

```
Java
filter(...)
```

nhận function đó làm đối số.

Dễ hiểu hơn

Bình thường:

```
Java
sum(3, 5)
```

→ truyền dữ liệu vào hàm.

Còn higher-order function:

```
Java
filter(x -> x > 5)
```

→ truyền logic / hành động vào hàm.

Tức là:

"Ế filter, đây là cách tao muốn mày lọc dữ liệu."



9.7. PARALLEL STREAMS

One of the biggest advantages of the Stream API and functional programming in general is the ability to easily write clear code to handle data in parallel. There's no need to manually create threads, check if the code is properly synchronized, and call **wait/notify methods**. All of this is done automatically within the parallel streams.

There are several simple ways to create parallel streams:

- `parallelStream()` method of a collection instead of `stream()` :

```
List<String> languages = List.of("java", "scala", "kotlin",  
                                "C#");  
  
List<String> jvmLanguages = languages.parallelStream()  
    .filter(lang -> !Objects.equals(lang, "C#"))  
    .collect(Collectors.toList());  
  
System.out.print(jvmLanguages); // [java, scala, kotlin]
```

- Convert an existing stream into a parallel stream using the `parallel()` method :

```
long sum = LongStream  
    .rangeClosed(1, 1_000_000)  
    .parallel()  
    .sum();  
  
System.out.println(sum); // 500000500000
```

It's actually easy to create a parallel stream, but should we always do this? Not necessarily; a parallel stream isn't always faster than its sequential equivalent. Several factors significantly affect the performance of a parallel stream. The number of CPU cores available at runtime significantly affects performance; more cores generally lead to better performance.

The cost per processing element is another important factor. The longer each element takes to process, the more efficient parallelization becomes, but parallel processing should not be used for excessively long operations (e.g., network connectivity).

Parallel chỉ ngon khi:

DATA LỚN
+
MỖI TASK KHẢ NĂNG



1. Bản chất thật sự

Sequential stream

```
Java  
numbers.stream()
```



Java làm:

1 thằng worker xử lý toàn bộ data



Parallel stream

```
Java  
numbers.parallelStream()
```



Java làm:

chia data cho nhiều worker



4. Tư duy cực quan trọng:

“Independent Tasks”

Parallel chỉ mạnh khi:

Mỗi phần tử độc lập nhau

Ví dụ:

```
Java  
.map(this::compressImage)
```



Ảnh A không liên quan ảnh B.

=> chia vô tư.

Nhưng nếu:

task này phụ thuộc task trước



=> parallel rất nguy hiểm.

5. Tư duy về ORDER

Đây là thứ nhiều người không hiểu.

Sequential

```
1 -> 2 -> 3 -> 4
```



Order tự nhiên.

Parallel

```
Thread 1 xử lý 1 2  
Thread 2 xử lý 3 4
```



Thằng nào xong trước xuất trước.

Vì vậy:

```
</> Java  
  
.forEach()
```



trong parallel:

KHÔNG đảm bảo thứ tự

Mental model

forEach

```
"ai xong trước nói trước"
```



forEachOrdered

```
"phải xếp hàng mới được nói"
```



Nhưng...

Nếu phải xếp hàng:

```
-> mất tốc độ
```



6. Tư duy reduce()

Đây là phần cực đáng học.

Sequential reduce

```
// Java
.reduce(100, Integer::sum)
```

Java nghĩ:

```
((100 + a) + b) + c)
```

Parallel reduce

Java nghĩ:

```
worker1: 100 + ...
worker2: 100 + ...
worker3: 100 + ...
```

💀 100 bị cộng nhiều lần.

Tư duy đúng:

Seed trong parallel phải "vô hại"

Neutral element là gì?

Là số:

thêm vào không đổi kết quả

Ví dụ cộng

$$x + 0 = x$$

=> neutral là 0.

Ví dụ nhân

$$x \times 1 = x$$

=> neutral là 1.



7. Tư duy quan trọng nhất: Shared Mutable State

Sai lầm kinh điển

```
</> Java  
  
List<Integer> result = new ArrayList<>();  
  
numbers.parallelStream()  
    .forEach(result::add);
```

Điều gì xảy ra?

Nhiều thread:

```
cùng sửa ArrayList
```

=> race condition.

Mental model

```
10 người cùng viết vào 1 cuốn sổ
```

=> ghi đè lung tung.

Vì vậy:

Parallel stream ghét mutable state.

Hãy dùng:

```
</> Java  
  
.collect(Collectors.toList())
```

vì collector được thiết kế cho parallel.

8. Parallel stream sinh ra cho cái gì?

CPU-bound tasks

Nghĩa là:

CPU phải "ngồi tính".

Ví dụ:

- xử lý ảnh
- encode video
- AI
- toán học
- matrix
- encryption

Nó KHÔNG sinh ra cho:

- database
- API
- network
- file I/O

Vì sao?

Ví thread sẽ:

đứng chờ



không dùng CPU.

12. Cheat sheet tư duy

Situation	Dùng gì
data nhỏ	stream
cần đúng order	stream
debug nhiều	stream
xử lý nặng	parallel
image/video/math	parallel
DB/API/network	thường không nên
shared variable	tránh parallel

list, map, tree

1. List

Interface lưu **danh sách có thứ tự**, cho phép trùng lặp.

```
Java
List<String> list = new ArrayList<>();
```

Đặc điểm

- Có index
- Cho phép duplicate
- Duyệt theo thứ tự thêm vào

Methods quan trọng

```
Java
add(E e)
get(int index)
set(int index, E e)
remove(int index)
remove(Object o)
contains(Object o)
size()
isEmpty()
clear()
```

2. ArrayList

Java Collections Framework

Implementation phổ biến nhất của `List`.

```
Java
ArrayList<String> arr = new ArrayList<>();
```

Cơ chế

- Dùng dynamic array bên dưới

Tốc độ

Operation	Speed
get(index)	O(1)
add cuối	O(1) amortized
insert giữa	O(n)
remove giữa	O(n)

Khi nào dùng?

- ✓ Truy cập bằng index nhiều
- ✓ Đọc dữ liệu nhiều hơn sửa

Ví dụ

```
Java
ArrayList<String> names = new ArrayList<>();

names.add("Bach");
names.add("An");

System.out.println(names.get(0));

names.remove("An");

for(String s : names){
    System.out.println(s);
}
```

3. LinkedList

`</>` Java



```
LinkedList<String> list = new LinkedList<>();
```

Cơ chế

- Doubly Linked List

Tốc độ

Operation	Speed
get(index)	O(n)
add đầu/cuối	O(1)
remove đầu/cuối	O(1)
insert giữa	O(n)

Khi nào dùng?

- ✓ Thêm/xóa đầu cuối nhiều
- ✗ Không phù hợp random access

Bonus

LinkedList implement cả:

- List
- Deque
- Queue

Nên dùng như queue được:

`</>` Java



```
list.addFirst("A");  
list.addLast("B");  
  
list.removeFirst();  
list.removeLast();
```

4. Set

Interface lưu dữ liệu không trùng lặp.

`Java`

```
Set<Integer> set = new HashSet<>();
```

Đặc điểm

- Không duplicate
- Không có index

5. HashSet

`Java`

```
HashSet<String> set = new HashSet<>();
```

Cơ chế

- Dựa trên hash table

Đặc điểm

- Không giữ thứ tự
- Rất nhanh

Tốc độ

Operation	Speed
add	O(1)
remove	O(1)
contains	O(1)

Ví dụ

`Java`

```
HashSet<String> set = new HashSet<>();

set.add("A");
set.add("B");
set.add("A");

System.out.println(set);
```

Output:

`Java`

[A, B]

6. TreeSet

`</> Java`



```
TreeSet<Integer> set = new TreeSet<>();
```

Cơ chế

- Red-Black Tree

Đặc điểm

- Tự sort tăng dần
- Không duplicate

Tốc độ

Operation	Speed
add	$O(\log n)$
remove	$O(\log n)$
contains	$O(\log n)$

Ví dụ

`</> Java`



```
TreeSet<Integer> nums = new TreeSet<>();

nums.add(5);
nums.add(1);
nums.add(3);

System.out.println(nums);
```

Output:

`</> Java`



```
[1, 3, 5]
```

7. Map

Interface lưu dữ liệu dạng:

key -> value



`</>` Java



```
Map<String, Integer> map = new HashMap<>();
```

Đặc điểm

- Key không trùng
- Value được trùng

Methods quan trọng

`</>` Java



```
put(key, value)  
get(key)  
remove(key)  
containsKey(key)  
containsValue(value)  
keySet()  
values()  
entrySet()
```

8. HashMap

`</>` Java



```
HashMap<String, Integer> map = new HashMap<>();
```

Cơ chế

- Hash Table

Đặc điểm

- Không giữ thứ tự
- Rất nhanh

Tốc độ

Operation	Speed
put	O(1)
get	O(1)
remove	O(1)

Ví dụ

`</>` Java



```
HashMap<String, Integer> scores = new HashMap<>();

scores.put("Bach", 10);
scores.put("An", 8);

System.out.println(scores.get("Bach"));
```

Duyệt Map

`</>` Java



```
for (Map.Entry<String, Integer> e : scores.entrySet()){
    System.out.println(e.getKey() + " " + e.getValue());
}
```

9. TreeMap

Java

```
TreeMap<String, Integer> map = new TreeMap<>();
```

Cơ chế

- Red-Black Tree

Đặc điểm

- Sort theo key

Ví dụ

Java

```
TreeMap<Integer, String> map = new TreeMap<>();
```

```
map.put(3, "C");  
map.put(1, "A");  
map.put(2, "B");
```

```
System.out.println(map);
```

Output:

Java

```
{1=A, 2=B, 3=C}
```

Quy tắc nhớ nhanh

Nếu cần:

Có index

→ `List`

Không duplicate

→ `Set`

key-value

→ `Map`

Trong implementation:

Nhanh nhất

→ `HashSet` , `HashMap`

Có sorting

→ `TreeSet` , `TreeMap`

Truy cập index nhanh

→ `ArrayList`

Thêm/xóa đầu cuối nhiều

→ `LinkedList`

Các phương thức thường xuyên xuất hiện

HashMap

//(key, value) = (String, String): Entry

//put get size isEmpty getKey getValue remove replace

HashSet

//add size contain remove addAll removeAll retainAll

LinkedHashMap

//(key, value) = (String, String): Entry

//put get size isEmpty getKey getValue remove replace

LinkedHashSet

//add size contain remove addAll removeAll retainAll

//LinkedHashSet có thứ tự xuất hiện

TreeMap

//(key, value) = (String, String): Entry

//put get size isEmpty getKey getValue remove replace

//firstKey lastKey firstEntry lastEntry headMap tailMap ceilingKey floorKey higherKey

lowerKey pollFirstEntry pollLastEntry

TreeSet

//add size contain remove addAll removeAll retainAll

// first last lower higher ceiling floor pollLast pollFirst headSet tailSet

//TreeSet có thứ tự

Phương thức Collections

Collections.sort(list);

Collections.reverse(list);

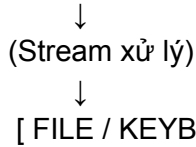
Collections.max(list);

Collections.min(list);

mấy cái IO

1. Luồng hoạt động I/O

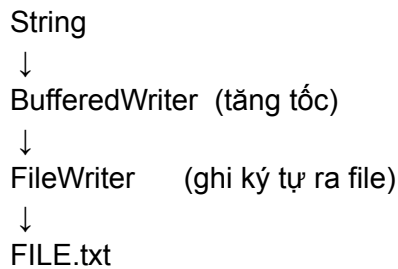
[DATA (Object / Text / Number)]



=> Mỗi class bạn hỏi chỉ là **một layer trong pipeline**

2. Luồng ghi FILE (WRITE)

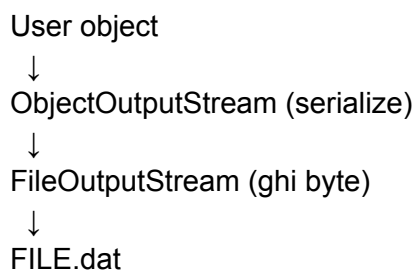
Ghi TEXT (chuẩn nhất ngoài đời)



=> Code tương ứng:

```
BufferedWriter bw = new BufferedWriter(new FileWriter("file.txt"));  
bw.write("hello");
```

Ghi OBJECT



👉 Ý nghĩa:

- Object → byte → lưu file

Ghi DATA (int, double,...)

int / String / double



DataOutputStream (format dữ liệu)



FileOutputStream



FILE.dat

3. Luồng đọc FILE (READ)

Đọc TEXT

FILE.txt



FileReader



BufferedReader (đọc nhanh + readLine)



String

👉 Code:

```
BufferedReader br = new BufferedReader(new FileReader("file.txt"));  
String line = br.readLine();
```

Đọc OBJECT

FILE.dat



FileInputStream



ObjectInputStream (deserialize)



User object



Đọc DATA

FILE.dat



FileInputStream



DataInputStream
↓
int / String / double

4. Luồng đặc biệt: bàn phím (System.in)

Đây là chỗ nhiều người “tắc não”

Keyboard (byte)
↓
InputStreamReader (byte → char)
↓
BufferedReader (đọc dễ hơn)
↓
String

=> Code:

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
```

5. Bản chất quan trọng (phải hiểu)

1. Stream = xếp lớp (wrap)

```
new BufferedReader(new FileReader("file.txt"))
```

nghĩa là:

BufferedReader
 bọc ngoài
FileReader

2. Mỗi lớp có nhiệm vụ riêng

Lớp	Nhiệm vụ
FileReader	đọc file
BufferedReader	tăng tốc
InputStreamReader	chuyển byte → char

ObjectStream	xử lý object
DataStream	xử lý kiểu dữ liệu

3. Nguyên tắc vàng

Luôn đi từ trong ra ngoài

File → Stream thấp → Stream cao → Data

6. So sánh cực dễ nhớ

Bạn muốn	Dùng
Đọc file text	BufferedReader + FileReader
Ghi file text	BufferedWriter + FileWriter
Lưu object	ObjectOutputStream
Lưu số liệu	DataOutputStream
Đọc bàn phím	BufferedReader + InputStreamReader